

NOV.24

SECURITY REVIEW REPORT FOR POLYGON

CONTENTS

- About Hexens
- Executive summary
 - Scope
- Auditing details
- Severity structure
 - Severity characteristics
 - Issue symbolic codes
- Findings summary
- Weaknesses
 - Inconsistency Between Comment and Implementation in Point at Infinity Check
 - Redundant branching in `dblScalarMulSecp256r1_scalar_table_k10_k20` table

ABOUT HEXENS

Hexens is a cybersecurity company that strives to elevate the standards of security in Web 3.0, create a safer environment for users, and ensure mass Web 3.0 adoption.

Hexens has multiple top-notch auditing teams specialized in different fields of information security, showing extreme performance in the most challenging and technically complex tasks, including but not limited to: Infrastructure Audits, Zero Knowledge Proofs / Novel Cryptography, DeFi and NFTs. Hexens not only uses widely known methodologies and flows, but focuses on discovering and introducing new ones on a day-to-day basis.

In 2022, our team announced the closure of a \$4.2 million seed round led by IOSG Ventures, the leading Web 3.0 venture capital. Other investors include Delta Blockchain Fund, Chapter One, Hash Capital, ImToken Ventures, Tensor Capital, and angels from Polygon and other blockchain projects.

Since Hexens was founded in 2021, it has had an impressive track record and recognition in the industry: Mudit Gupta - CISO of Polygon Technology - the biggest EVM Ecosystem, joined the company advisory board after completing just a single cooperation iteration. Polygon Technology, 1inch, Lido, Hats Finance, Quickswap, Layerswap, 4K, RociFi, as well as dozens of DeFi protocols and bridges, have already become our customers and taken proactive measures towards protecting their assets.

SCOPE

The analyzed resources are located on:

<https://github.com/0xPolygonHermes/zkevm-rom/pull/409/files>

<https://github.com/0xPolygonHermes/zkevm-rom/compare/v9.0.0-rc.1-fork.13...v9.0.0-rc.2-fork.13>

The issues described in this report were fixed in the following commit:

<https://github.com/0xPolygonHermes/zkevm-rom/pull/421/commits/b96e2cc8df4208c1e16e6b8bcef35d8cb943cea9>

AUDITING DETAILS



STARTED
18.11.2024

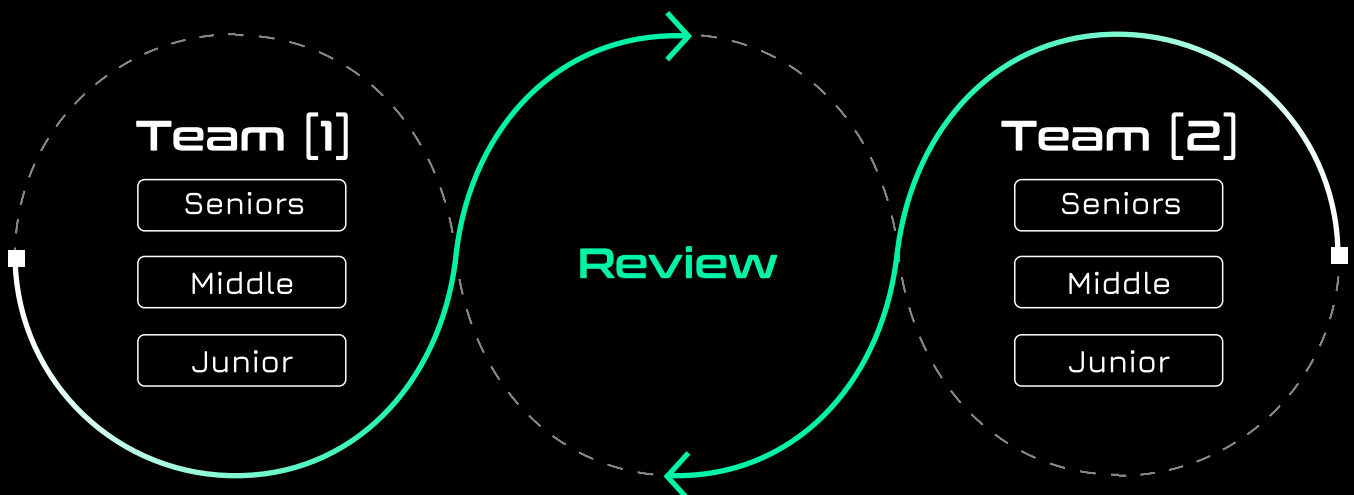
Review
Led by

DELIVERED
23.12.2024

**HAYK
ANDRIASYAN**
Senior Security
Researcher | Hexens

HEXENS METHODOLOGY

Hexens methodology involves 2 teams, including multiple auditors of different seniority, with at least 5 security engineers. This unique cross-checking mechanism helps us provide the best quality in the market.



SEVERITY STRUCTURE

The vulnerability severity is calculated based on two components

- Impact of the vulnerability
- Probability of the vulnerability

Impact	Probability			
	rare	unlikely	likely	very likely
Low/Info	Low/Info	Low/Info	Medium	Medium
Medium	Low/Info	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

SEVERITY CHARACTERISTICS

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities with this level of severity can result in significant financial losses or reputational damage. They often allow an attacker to gain complete control of a contract, directly steal or freeze funds from the contract or users, or permanently block the functionality of a protocol. Examples include infinite mints and governance manipulation.

High

Vulnerabilities with this level of severity can result in some financial losses or reputational damage. They often allow an attacker to directly steal yield from the contract or users, or temporarily freeze funds. Examples include inadequate access control integer overflow/underflow, or logic bugs.

Medium

Vulnerabilities with this level of severity can result in some damage to the protocol or users, without profit for the attacker. They often allow an attacker to exploit a contract to cause harm, but the impact may be limited, such as temporarily blocking the functionality of the protocol. Examples include uninitialized storage pointers and failure to check external calls.

Low

Vulnerabilities with this level of severity may not result in financial losses or significant harm. They may, however, impact the usability or reliability of a contract. Examples include slippage and front-running, or minor logic bugs.

Informational

Vulnerabilities with this level of severity are regarding gas optimizations and code style. They often involve issues with documentation, incorrect usage of EIP standards, best practices for saving gas, or the overall design of a contract. Examples include not conforming to ERC20, or disagreement between documentation and code.

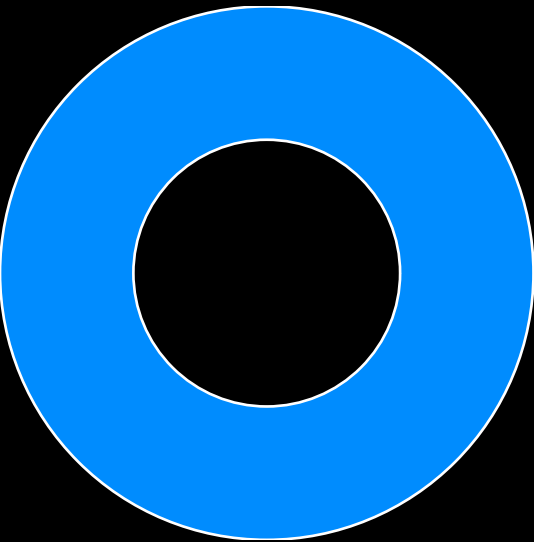
ISSUE SYMBOLIC CODES

Every issue being identified and validated has its unique symbolic code assigned to the issue at the security research stage. Cause of the vulnerability reporting flow design, some of the rejected issues could be missing.

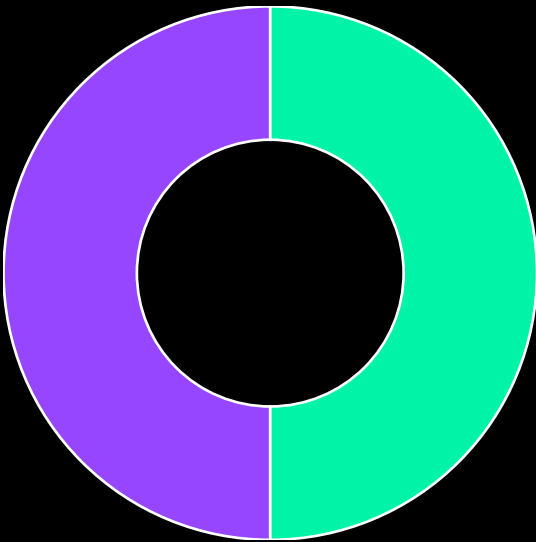
FINDINGS SUMMARY

Severity	Number of Findings
Critical	0
High	0
Medium	0
Low	0
Informational	2

Total: 2



• Informational



• Fixed
• Acknowledged

WEAKNESSES

This section contains the list of discovered weaknesses.

ZKEPOL-1

INCONSISTENCY BETWEEN COMMENT AND IMPLEMENTATION IN POINT AT INFINITY CHECK

SEVERITY: **Informational**

PATH:

main/p256verify/p256verify.zkasm#L104

REMEDIATION:

If keeping the check, update the comment to accurately reflect the implementation.

STATUS: **Fixed**

DESCRIPTION:

The comment in the p256verify function suggests that it checks whether the public key is the point at infinity ($\mathcal{O}$). However, the actual implementation checks if the point is $(0,0)$.

- [RIP-7212](#) specifically notes: "many implementations use $(0, 0)$ as the reference point at infinity, which is not on the curve and should therefore be rejected"
- The check for $(0,0)$ is redundant as it would be caught by the subsequent curve equation verification.

This inconsistency between the comment and implementation could lead to confusion during code maintenance or future updates.

```

; 5] Check whether PK = (x,y) is the point at infinity 0
0n => B
$ => A      :MLOAD(p256verify_x)
$           :EQ, JMPNC(p256verify_pk_not_point_at_infinity)
$ => A      :MLOAD(p256verify_y)
$           :EQ, JMPC(p256verify_error_pk_point_at_infinity)

```

REDUNDANT BRANCHING IN DBLSCALARMULSECP256R1_SCALAR_ TABLE_K10_K20 TABLE

SEVERITY: **Informational**

PATH:

main/p256verify/dblScalarMulSecp256r1.zkasm

REMEDIATION:

Change

JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR) to JMP(dblScalarMulSecp256r1_double_noRR) in dbScalarMulSecp256r1_scalar_table_k10_k20 table.

STATUS: **Acknowledged**

DESCRIPTION:

secp256r1 curve based ECDSA signature verification includes elliptic curve operations: $[k1] \cdot P1 + [k2] \cdot P2$ which is handled by **dblScalarMulSecp256r1**. Elliptic curve multiplications are done via **double-and-add** method which is based on the binary representation of the scalar values.

dblScalarMulSecp256r1 contains precomputed tables **dblScalarMulSecp256r1_scalar_table_k10_k20**, **dblScalarMulSecp256r1_scalar_table_k10_k21**, **dblScalarMulSecp256r1_scalar_table_k11_k20**, **dblScalarMulSecp256r1_scalar_table_k11_k21** for each case of **k1** and **k2** MSBs.

In a case of **k1** and **k2** MSBs are 0, zkEVM jumps to the **dblScalarMulSecp256r1_scalar_table_k10_k20** table:

```

; high_bit(k1) == 0 high_bit(k2) == ??
dblScalarMulSecp256r1_k10:
    ; Receive the next MSB bit of k2
    $0{(mem.dblScalarMulSecp256r1_k2) >> RR & 0x1}
:JMPZ(@dblScalarMulSecp256r1_scalar_table_k10_k20 + RR)
                                :JMP(@dblScalarMulSecp256r1_scalar_table_k10_k21 + RR)

```

The application jumps on the line which is currently assigned in the **RR** registry.

```

dblScalarMulSecp256r1_scalar_table_k10_k20:
    $ => C          :MLOAD(dblScalarMulSecp256r1_acum_k1), JMP(dblScalarMulSecp256r1_save_k10_k20) ; RR = 0
    RR - 1 => RR     :JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR)      ; RR = 1
    RR - 1 => RR     :JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR)      ; RR = 2
    RR - 1 => RR     :JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR)      ; RR = 3
    RR - 1 => RR     :JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR)      ; RR = 4
    RR - 1 => RR     :JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR)      ; RR = 5
    RR - 1 => RR     :JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR)      ; RR = 6
    RR - 1 => RR     :JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR)      ; RR = 7
    RR - 1 => RR     :JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR)      ; RR = 8
    ...

```

In every line the **RR** value has a non negative value after **RR - 1 => RR** operation, so it can't be a negative number and

```
JMPN(dblScalarMulSecp256r1_end_loop,dblScalarMulSecp256r1_double_noRR)
```

true branch will never be called.

hexens x  polygon